

# Chapter 2 Lab – Using Embeddings

## Advanced Machine Learning for Finance

Giovanni Della Lunga

Master's Degree in Quantitative Finance  
University of Bologna

Bologna, 2026

# Section 1

## Using Pre-trained Vectors

**Notebooks:** 01\_using\_pretrained\_vectors.ipynb  
02\_evaluating\_embeddings.ipynb

# Notebook Overview

## What this notebook does

This notebook loads, evaluates, and visualises pre-trained GloVe word vectors using Gensim. It is a revised and updated version of a workflow originally developed by Stefan Jansen in *Machine Learning for Algorithmic Trading*, adapted to current library versions (Gensim 4, pandas  $\geq 2$ ).

The notebook is organised into four sequential steps:

- 1 **Format conversion** — translate GloVe text files into the word2vec-compatible format expected by Gensim.
- 2 **File inspection** — verify structural integrity before loading a multi-gigabyte embedding file.
- 3 **Analogy evaluation** — measure how well the embeddings solve word analogy tasks across three training corpora.
- 4 **Visualisation** — project the 300-d vectors to 2D with PCA and plot geometric analogy relationships.

## Honest scope

The notebook works with *general-purpose* GloVe vectors. Their suitability for financial NLP tasks is critically assessed at the end.

Three pre-trained variants are used in the notebook:

| Corpus       | Tokens | Vocab | Dim |
|--------------|--------|-------|-----|
| Wikipedia    | 6B     | 400K  | 300 |
| Twitter      | 27B    | 1.2M  | 200 |
| Common Crawl | 840B   | 2.2M  | 300 |

# Static Embeddings: One Word, One Vector

## The fundamental constraint

Each word type is assigned **exactly one** vector, regardless of context. Polysemous words receive a single representation that averages across all their senses.

This matters in finance:

- ▶ bank → financial institution *or* river bank
- ▶ spread → credit spread *or* bid–ask spread *or* butter on toast
- ▶ call → call option *or* phone call

## Domain mismatch compounds the problem

GloVe vectors trained on Wikipedia or Common Crawl reflect *general* English usage. In those corpora, “bond” is nearly as likely to refer to James Bond as to a fixed-income instrument. Financial downstream tasks therefore often require either domain-specific training or contextual models (BERT, FinBERT) that can resolve sense from context.

# Gensim: A Python Library for Vector Space Modelling

## What Gensim is

Gensim is an open-source Python library designed for unsupervised learning of vector representations from raw text. Its core abstraction is the **KeyedVectors** object: a mapping from word strings to dense numerical vectors that supports vocabulary lookup, similarity queries, and analogy arithmetic.

In this notebook we use Gensim for three specific operations:

- ▶ **Loading** large pre-trained embedding files in word2vec-compatible format via `KeyedVectors.load_word2vec_format()`.
- ▶ **Evaluating** embeddings on word analogy benchmarks via `model.evaluate_word_analogies()`, which implements the vector-arithmetic test  $\mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c \approx \mathbf{v}_d$ .
- ▶ **Extracting** the raw vector matrix and the `index_to_key` vocabulary list for downstream processing (normalisation, PCA, plotting).

# Gensim: A Python Library for Vector Space Modelling

## API change: Gensim 3 → Gensim 4

The original Jansen notebook was written for Gensim 3. In Gensim 4 the attribute `index2word` was renamed `index_to_key` and several model methods were reorganised. The notebook has been updated accordingly; running it on Gensim 3 will raise `AttributeError`.

## Step 1 — Format Conversion: GloVe → word2vec

GloVe text files and word2vec text files are nearly identical. The only structural difference is a **header line**:

**GloVe format** (no header):

```
king    0.123  -0.084  0.532  ...
queen   0.221   0.017  0.418  ...
man     0.091  -0.140  0.302  ...
```

**word2vec format** (with header):

```
400000 300
king    0.123  -0.084  0.532  ...
queen   0.221   0.017  0.418  ...
man     0.091  -0.140  0.302  ...
```

The notebook converts using `glove2word2vec` (Gensim utility):

```
from gensim.scripts.glove2word2vec import glove2word2vec
glove2word2vec(glove_input_file=glove_wiki_file ,
               word2vec_output_file=word2vec_wiki_file)
```



## Step 2 — Inspecting the Embedding File Before Loading

Large embedding files (Common Crawl:  $\approx 5.4$  GB) are expensive to reload. A diagnostic check before calling Gensim prevents confusing errors.

The notebook implements `inspect_word_vector_file()`, which:

- ▶ Reads the header line and verifies it contains two integers (*vocabulary size*, *dimension*).
- ▶ Counts the actual number of data lines and compares with the header declaration.
- ▶ Identifies **bad lines** — those with a field count different from  $d + 1$  (one token plus  $d$  floats).

## Step 3 — Analogy Evaluation

### The analogy task

A standard intrinsic benchmark for word embeddings. Given three words  $a$ ,  $b$ ,  $c$ , the model must retrieve the word  $d$  such that:

$$a : b :: c : d \iff \mathbf{v}_b - \mathbf{v}_a + \mathbf{v}_c \approx \mathbf{v}_d$$

The notebook uses `model.evaluate_word_analogies()`, which evaluates 14 categories covering both semantic and syntactic relationships.

#### Semantic categories

- ▶ Capital cities and countries
- ▶ Currency pairs
- ▶ Family relationships

#### Syntactic categories

- ▶ Adjective  $\rightarrow$  adverb
- ▶ Comparative / superlative
- ▶ Past tense, plural forms

## Step 3 — Analogy Results Across Three Corpora

| Category                | Twitter      | Wikipedia    | Common Crawl |
|-------------------------|--------------|--------------|--------------|
| Capital cities (common) | 70.1%        | 94.9%        | 94.7%        |
| Capital cities (world)  | 69.0%        | 96.5%        | 91.7%        |
| City in state           | 35.1%        | 60.0%        | 70.7%        |
| <b>Currency</b>         | <b>1.9%</b>  | <b>17.4%</b> | <b>18.4%</b> |
| Family relationships    | 82.5%        | 88.1%        | 97.9%        |
| Adjective → adverb      | 14.3%        | 22.6%        | 38.8%        |
| Comparative             | 75.8%        | 88.2%        | 87.7%        |
| Past tense              | 57.6%        | 61.2%        | 62.1%        |
| <b>Total accuracy</b>   | <b>56.4%</b> | <b>75.4%</b> | <b>77.9%</b> |

### Currency analogies fail across all corpora

The “currency” category achieves at most 18% accuracy even on the largest corpus. Currency relationships are sparse, geographically specific, and poorly represented in general web text. This is precisely the kind of domain gap that financial NLP practitioners must account for.

# Training Corpus Matters More Than Corpus Size

## Wikipedia $\approx$ Common Crawl $\gg$ Twitter

Despite Twitter having  $4.5\times$  the token count of Wikipedia, its embeddings score 19 percentage points lower overall. Informal writing, abbreviations, misspellings, and non-standard grammar prevent the model from learning clean co-occurrence statistics.

### Why Wikipedia works well

- ▶ Encyclopaedic, factual prose.
- ▶ Consistent named-entity usage (capitals, countries, currencies).
- ▶ Clean tokenisation.

### Why Common Crawl adds value

- ▶ Larger vocabulary (2.2M vs. 400K).
- ▶ Better coverage of rare entities.
- ▶ City-in-state: +11 pp over Wikipedia.

# Training Corpus Matters More Than Corpus Size

## Implication for quantitative finance

For financial text tasks, the appropriate starting point is embeddings trained on financial corpora (10-K filings, earnings calls, Bloomberg news) — not general web text. This motivates FinBERT and similar domain-adapted models, which will be covered in Lesson 3.

## Step 4 — Projecting to 2D with Incremental PCA

The notebook reduces the 300-dimensional Wikipedia embedding matrix ( $100,000 \times 300$ ) to 2D using `IncrementalPCA` from `scikit-learn`, which avoids loading the full matrix into memory at once.

```
from sklearn.decomposition import IncrementalPCA
from numpy.linalg import norm

# L2-normalise vectors before projection
vectors = model.vectors.copy()
vectors = vectors / norm(vectors, axis=1, keepdims=True)

pca = IncrementalPCA(n_components=2)
vectors2D = pca.fit_transform(vectors)
```

## Step 4 — Projecting to 2D with Incremental PCA

### How much variance does 2D retain?

With 300-dimensional GloVe vectors, the first two principal components typically explain only **2–4%** of total variance. The projection is therefore highly lossy. It is useful for visualising directional relationships between selected word groups, but should not be interpreted as a faithful map of the full embedding space.

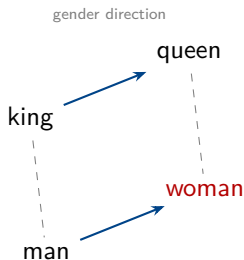
## Step 4 — Visualising Analogy Geometry

For each analogy category, the notebook selects the **most representative example** — the correctly solved analogy whose two displacement vectors are most parallel in 2D:

$$\mathbf{v}_b - \mathbf{v}_a \parallel \mathbf{v}_d - \mathbf{v}_c$$

Parallelism is measured by cosine similarity of the two displacement vectors after PCA projection.

The resulting plot ( $3 \times 4$  grid, one panel per category) shows arrows from  $a \rightarrow b$  and from  $c \rightarrow d$ . When the embedding correctly captures the relationship, the two arrows point in the same direction.



Parallel arrows indicate that the vector offset encoding a relationship is *consistent* across different word pairs.



# Critical Assessment: What This Notebook Does and Does Not Show

## What the notebook demonstrates effectively

- ▶ Analogy accuracy is a reproducible and interpretable metric for comparing embedding corpora.
- ▶ Training corpus quality and domain coverage dominate corpus size.
- ▶ Geometric linearity of semantic relationships is real and visually verifiable in the 2D projection.

# Critical Assessment: What This Notebook Does and Does Not Show

## What the notebook does not tell us

- ▶ **Analogy accuracy  $\neq$  downstream task performance.** High analogy scores do not guarantee good sentiment classification, NER, or sequence modelling.
- ▶ **Currency and finance-domain analogies fail.** The benchmark confirms the domain gap rather than resolving it.
- ▶ **The 2D visualisation is highly lossy.** Apparent non-parallelism in plots may reflect projection artefacts, not true embedding failures.
- ▶ **Static embeddings cannot handle polysemy.** A downstream task on central bank communications would need contextual representations (BERT, FinBERT) to correctly distinguish, for instance, the monetary and physical senses of “tightening”.

## Four-step workflow

- ➊ Convert GloVe  $\rightarrow$  word2vec format (add vocabulary/dimension header)
- ➋ Inspect file structure before loading (validate header, count lines, flag bad tokens)
- ➌ Evaluate on word analogy benchmark across Twitter, Wikipedia, Common Crawl
- ➍ Visualise geometry via PCA projection and displacement parallelism plots

## Key takeaways

- ▶ Wikipedia and Common Crawl vectors substantially outperform Twitter vectors.
- ▶ Currency analogies fail everywhere — a direct consequence of domain mismatch.
- ▶ Analogy evaluation is a useful but incomplete proxy for downstream utility.
- ▶ Static embeddings are the natural starting point for understanding dense representations; their limitations motivate contextual models covered in Lesson 3.

*Notebook Reference:* Jansen (2020), *ML for Algorithmic Trading*, Chapter 16 on Word Embeddings

## Section 2

# Financial News Pre-Processing

**Notebooks:** `03_financial_news_preprocessing`

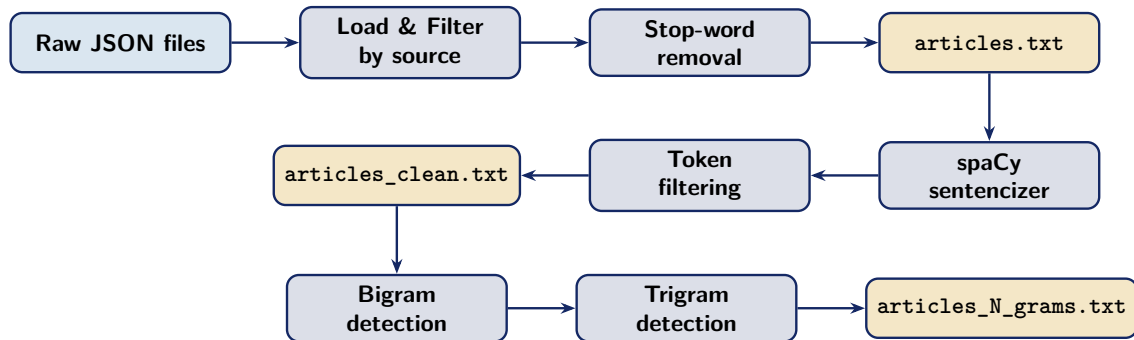
## Why preprocess financial news?

- ▶ Word embedding models (Word2Vec, GloVe) require **clean, tokenised, lower-cased** text.
- ▶ Raw news feeds contain HTML entities, numbers, punctuation and boilerplate that **corrupt the embedding space**.
- ▶ Financial language is domain-specific: multi-word expressions such as *central\_bank* or *interest\_rate* carry atomic meaning and must be preserved as **n-grams**.

## Data source

- ▶ **US Financial News** dataset (JSON archive)
- ▶ Sources: CNBC, Reuters (Business, World, Technology, Money), WSJ
- ▶ Each article stored as a JSON object with `thread.section_title` and `text` fields

# End-to-End Preprocessing Pipeline



## Stage 1 — Ingest

JSON parsing, source filtering,  
initial stop-word stripping

## Stage 2 — Clean

Sentence segmentation, token-level  
quality filtering

## Stage 3 — N-grams

Statistical collocation detection  
(Gensim Phrases)

# Stage 1 — Data Ingestion & Source Filtering

## Trusted section titles (11 sources)

- ▶ Press Releases — CNBC
- ▶ Reuters: Company / World / Business News
- ▶ Reuters: Financial Services & Real Estate
- ▶ Reuters: Money / Technology News
- ▶ Top News and Analysis (pro)
- ▶ The Wall Street Journal — Breaking News

## Stop-word list

Downloaded from the Glasgow IR Resources server (~571 common English words). Applied at **word-token level** before saving to `articles.txt`.

```
def read_articles():
    articles, counter = [], Counter()
    for f in data_path.glob('*/**/*.json'):
        with f.open("r", encoding="utf-8")
            as fp:
                article = json.load(fp)
                if article['thread']['section_title']
                    in set(section_titles):
                    text = article['text'].lower()
                        .split()

                    counter.update(text)
                    articles.append(
                        ' '.join([t for t in text
                                if t not in stop_words])
                    )
    return articles, counter
```

**Note:** explicit utf-8 encoding prevents Windows cp1252 errors.

## Stage 2 — Sentence Segmentation & Token Filtering

### spaCy pipeline

- ▶ Blank English model (`spacy.blank("en")`)
- ▶ **Sentencizer** component added: rule-based, fast, no POS/NER overhead
- ▶ Batched inference via `nlp.pipe` (`batch_size=100, n_process=8`)

### Sentence quality gates

A sentence is **kept** if and only if:

- ▶  $\geq 6$  valid tokens after filtering
- ▶  $< 100$  raw spaCy tokens (removes boilerplate “mega-sentences”)

```
def clean_doc(d):
    doc = []
    for sent in d.sents:
        s = [
            t.text.lower()
            for t in sent
            if not any([
                t.is_digit,
                not t.is_alpha,
                t.is_punct,
                t.is_space
            ])
        ]
        if len(s) > 5 and len(sent) < 100:
            doc.append(' '.join(s))
    return doc
```



## What is measured

After writing `articles_clean.txt`, the notebook computes:

- ▶ **Vocabulary size:** unique tokens (further filtered against stop-words)
- ▶ **Sentence-length distribution:**  
`pd.Series.describe` with decile percentiles  
(`np.arange(.1, 1, .1)`)
- ▶ **Top-25 tokens:** horizontal bar chart (most frequent non-stop-word types)

## Key metrics (illustrative)

| Metric                 | Value                    |
|------------------------|--------------------------|
| Sentences (raw)        | $N$ articles             |
| Sentences (clean)      | $< N$ (quality filter)   |
| Median sentence length | $\sim 12$ tokens         |
| Vocabulary (filtered)  | $\sim$ tens of thousands |

Exact counts depend on the corpus snapshot. The notebook prints live counts at runtime.

## Stage 3 — Statistical N-Gram Detection

### Gensim Phrases / Phraser

- ▶ **Iterative:** bigrams first, then trigrams built on top of bigrams
- ▶ **Scoring hyper-parameters:**  
threshold=100, min\_count=10
- ▶ A phrase ( $w_i, w_{i+1}$ ) is promoted when its collocation score exceeds the threshold
- ▶ Joined with underscore:  
interest\_rate, central\_bank

```
sentences = LineSentence(clean_article_path)

for n in range(2, max_length + 1):    # 2, 3
    if n > 2:
        sentences = LineSentence(
            results_path /
            f'articles_{n-1}_grams.txt'
        )
    phrases = Phrases(sentences,
                      threshold=100,
                      min_count=10)
    phrase_scores = phrases
                      .find_phrases(sentences)
    grams = Phraser(phrases)
    sentences = grams[sentences]
    with open(f'articles_{n}_grams.txt', 'w')
        as f:
        for sent in sentences:
            f.write(' '.join(sent) + '\n')
```

# Key Libraries & Design Choices

## Library stack

| Library        | Role                          |
|----------------|-------------------------------|
| <b>spaCy</b>   | Sentence segmentation         |
| <b>Gensim</b>  | Phrase detection & corpus I/O |
| <b>pandas</b>  | Stats & top-token ranking     |
| <b>Seaborn</b> | Frequency visualisation       |
| <b>pathlib</b> | Cross-platform path handling  |

## Reproducibility

- ▶ `np.random.seed(42)` set globally
- ▶ All file I/O uses explicit `utf-8` encoding

## Design decisions

- ▶ **Blank spaCy model:** avoids loading the full statistical pipeline (POS, NER) which is unnecessary for sentence splitting — significantly faster on large corpora
- ▶ **LineSentence:** memory-efficient streaming from disk; no need to load the full corpus into RAM
- ▶ **Two-pass n-gram building:** ensures trigrams subsume coherent bigrams, giving semantically richer compound tokens

## What this notebook delivers

- 1 **Filtered corpus:** only articles from trusted financial news sections (CNBC, Reuters, WSJ)
- 2 **Clean sentence corpus:** lower-cased, alphabetic-only tokens, quality-gated sentences
- 3 **N-gram corpus** (up to trigrams): domain-relevant collocations preserved as atomic units
- 4 **Corpus statistics:** vocabulary size, sentence-length distribution, top-token frequency chart

## Downstream use

The file `articles_3_grams.txt` is the direct input to **Word2Vec training**, enabling the model to learn embeddings that capture financial collocations as single vectors.

## Artefacts produced

|                                   |                       |
|-----------------------------------|-----------------------|
| <code>articles.txt</code>         | Raw (stop-word filt.) |
| <code>articles_clean.txt</code>   | Sentence-cleaned      |
| <code>articles_2_grams.txt</code> | Bigram corpus         |
| <code>articles_3_grams.txt</code> | Trigram corpus        |

*“Good embeddings start with good text cleaning.”*

## Section 3

# Training Word2Vec on Financial News

**Notebooks:** 04\_financial\_news\_word2vec\_tensorflow.ipynb  
05\_financial\_news\_word2vec\_gensim.ipynb

# Why Train Custom Financial Embeddings?

- ▶ Generic pre-trained models (GloVe, Word2Vec/Wikipedia) **miss financial vocabulary**: *quantitative easing, yield\_curve, earnings\_guidance, ...*
- ▶ Domain language **shifts over time** — new products, regulations and crises continuously introduce unseen words.
- ▶ Out-of-vocabulary words receive a **default zero-information vector**, degrading all downstream tasks.
- ▶ Corporate earnings releases and Reuters wire stories use **nuanced co-occurrence patterns** that general corpora do not capture.

## Solution

Train Word2Vec **from scratch** on the preprocessed financial news corpus produced by Notebook 03.

# Shared Setup Across Both Notebooks

## Common inputs

- ▶ Corpus: `articles_3_grams.txt` (output of NB 03)
- ▶ Algorithm: **Word2Vec Skip-gram**
- ▶ Evaluation: Google word-analogy benchmark (`analogies-en.txt`)

## Common hyperparameters

| Parameter           | Value             |
|---------------------|-------------------|
| Embedding dimension | 300               |
| N-gram order        | 3 (trigrams)      |
| Loss                | Negative Sampling |

## Two complementary perspectives

- ▶ **NB 04 (Keras/TF)**: builds Word2Vec *from scratch* to expose every architectural detail.
- ▶ **NB 05 (Gensim)**: uses an optimised C-level backend for fast, production-grade training.

# The Skip-Gram Model

## Core objective

Given target word  $w_t$ , predict surrounding context within window  $k$ :

$$\max \sum_{-k \leq j \leq k, j \neq 0} \log P(w_{t+j} \mid w_t)$$

## Negative Sampling

Replaces softmax  $\mathcal{O}(V)$  with a binary classifier that distinguishes **real pairs** from **noise pairs** — cost drops to  $\mathcal{O}(k_{\text{neg}})$ .

## Subsampling

Frequent words downsampled with  $P(w) = 1 - \sqrt{t/f(w)}$ , balancing rare and common tokens.



## Token-to-ID mapping

- 1 Keep tokens with  $\text{MIN\_FREQ} \geq 10$
- 2 Assign integer IDs; unknown  $\rightarrow 0$
- 3 Drop sentences  $> 50$  tokens
- 4 Sample 50% of sentences

## Pair generation

Keras `skipgrams()` produces  
(target, context, label) triples.

Persisted to `data.h5` to avoid regeneration  
across runs.

```
# Vocabulary
token_counts = [t for t in
    Counter(words).most_common()
    if t[1] >= MIN_FREQ]
tokens, _ = zip(*token_counts)
id_to_token = dict(enumerate(tokens, 1))
id_to_token[0] = 'UNK'
```

```
# Skip-gram pairs
pairs, labels = skipgrams(
    sequence          = data,
    vocabulary_size   = vocab_size,
    window_size       = WINDOW_SIZE,
    sampling_table     = sampling_table,
    negative_samples= 1.0,
    shuffle            = True
)
```

### Functional Keras graph

- 1 Two scalar Input layers
- 2 **Shared** Embedding ( $V \times 300$ )
- 3 Dot product of the two vectors
- 4 Dense(1, sigmoid) output
- 5 Loss: binary\_crossentropy  
Opt: RMSprop

### Validation model

Same weights, cosine similarity.  
Nearest-neighbour checks and analogy  
accuracy run at each epoch.

```
embedding = Embedding(vocab_size ,  
                      EMBEDDING_SIZE, input_length=1)  
  
target = Reshape((EMBEDDING_SIZE, 1))(  
            embedding(input_target))  
context = Reshape((EMBEDDING_SIZE, 1))(  
            embedding(input_context))  
  
dot = Dot(axes=1)([target , context])  
out = Dense(1, activation='sigmoid')(dot)  
  
model = Model(  
    inputs = [input_target , input_context] ,  
    outputs = out)  
model.compile(  
    loss = 'binary_crossentropy' ,  
    optimizer = 'rmsprop')
```

## Training loop

- ▶ `EPOCHS = 1, BATCH_SIZE = 2500`
- ▶ `EvalCallback` runs at train start, epoch end, and train end:
  - Prints  $k$ -nearest neighbours for 10 validation words
  - Reports analogy accuracy at top-1, top-5 and top-10
- ▶ Model saved as `skipgram_model.h5`

## TensorBoard integration

- ▶ Token metadata exported to `meta.tsv`
- ▶ TensorBoard callback writes embedding projector logs
- ▶ Enables **interactive 3-D visualisation** of the 300-dimensional embedding space directly in the browser

## Pedagogical value

The explicit Keras graph makes every component of Word2Vec directly inspectable and modifiable — ideal for understanding the architecture.

## Single-call training

The entire pipeline — vocabulary building, subsampling, negative sampling, multi-threaded SGD — is wrapped in one constructor:

```
model = Word2Vec(  
    sentences = LineSentence(sentence_path)  
    sg        = 1,          # Skip-gram  
    vector_size= 300,  
    window    = 5,  
    min_count  = 100,  
    negative   = 20,  
    workers    = 8,  
    epochs     = 1,  
    alpha      = 0.05  
)
```

## Key differences vs NB 04

- ▶ `min_count` = 100 (vs 10 in NB 04) — keeps only well-attested words
- ▶ `negative` = 20 noise samples (vs 1) — much stronger gradient signal
- ▶ `window` = 5 (vs 3)
- ▶ `workers` = 8 — true multi-threaded C backend
- ▶ `LineSentence` streams from disk — no full corpus loaded into RAM

### Training loop (up to 10 epochs)

After the initial epoch, the model is trained for up to 9 additional epochs via `model.train()`.

After each epoch:

- ▶ Analogy accuracy is evaluated with `evaluate_word_analogies()`
- ▶ Model is **checkpointed only when accuracy improves** (best-model tracking)
- ▶ Accuracy history saved to `accuracies.csv`

```
for i in range(1, 10):
    model.train(
        sentences,
        epochs=1,
        total_examples=model.corpus_count
    )
    score, details = model.wv\
        .evaluate_word_analogies(
            analogy_path,
            case_insensitive=True
        )
    if score > best_accuracy:
        model.save(f"word2vec_{i}.model")
        model.wv.save(
            f"word_vectors_{i}.kv")
        best_accuracy = score
```

### Analogy benchmark — 14 categories

- ▶ *Semantic*: capitals, currencies, family relations, city-state pairs
- ▶ *Syntactic*: comparatives, superlatives, past tense, plurals, present participle, ...

Results shown per category as a stacked bar chart (correct vs incorrect) with accuracy overlay.

### Qualitative checks

- ▶ **Classic analogy:**  
 $king - man + woman \approx queen$
- ▶ **Most-similar lookup:** for 10 randomly sampled frequent tokens, print the 10 nearest neighbours in cosine space
- ▶ Results exported to `most_similar.csv`

# NB 04 vs NB 05 — Side-by-Side Comparison

## NB 04 — Keras / TensorFlow

- ▶ **Purpose:** expose architecture step by step
- ▶ **Pairs:** manual via `skipgrams()`
- ▶ **Negatives:** 1 sample
- ▶ **Epochs:** fixed at 1
- ▶ **Speed:** slower — Python-level TF graph
- ▶ **Viz:** TensorBoard 3-D projector
- ▶ **Output:** `skipgram_model.h5`

## NB 05 — Gensim

- ▶ **Purpose:** fast, production-grade training
- ▶ **Pairs:** handled internally by Gensim
- ▶ **Negatives:** 20 samples
- ▶ **Epochs:** up to 10, accuracy checkpointing
- ▶ **Speed:** faster — multi-threaded C backend
- ▶ **Viz:** Matplotlib bar charts + CSV
- ▶ **Output:** `word2vec.model + .kv`

Same algorithm, same goal — Keras reveals *how* it works; Gensim reveals *how well* it can be trained at scale.

# Key Takeaways

- 1 **Domain specificity matters.** Training on financial news produces embeddings that correctly cluster *yield*, *rate*, *bond* and related terms — not possible with generic GloVe vectors.
- 2 **N-grams add value.** Trigram pre-processing (NB 03) ensures that *central\_bank* and *interest\_rate* are treated as single atomic tokens.
- 3 **Keras = transparency; Gensim = performance.** Use NB 04 to understand the model; use NB 05 to train it seriously.
- 4 **Analogy accuracy as a relative signal.** The benchmark is imperfect for financial text, but provides a consistent metric to compare configurations and select the best epoch checkpoint.